Execution Complexity Signatures: Predicting Autonomous Coding Task Success from Early-Phase Behavioral Patterns

Straughter Guthrie
Viable Systems Research

Abstract

Autonomous software engineering agents exhibit highly variable task completion rates, yet current systems lack mechanisms to predict failures early enough for effective intervention. We investigate whether execution patterns observable during planning and early implementation phases can predict task outcomes before significant resources are consumed. Analyzing 208 autonomous coding task executions across 12 software projects, we train binary classifiers at two decision points: immediately after planning (using task metadata, planning artifacts, and story counts) and after 25% of implementation iterations (adding behavioral signals like iteration velocity and duration trends). Our dataset includes tasks executed with two merge strategies (direct and pull request modes) and three agent configurations, achieving an overall 55.8% success rate. Logistic regression and random forest models trained solely on post-planning features achieved cross-validated AUC scores of 0.935 and 0.927, with planning completeness (presence of PRDs, research artifacts) and task scope (story counts) emerging as the strongest predictors. Incorporating early-implementation behavioral signals yielded comparable performance, indicating that static task properties already capture most information about task tractability. Critically, merge mode showed a strong association with outcomes (75.0% success for direct merge vs 40.5% for pull requests, χ^2=23.35, p$<$0.001), and 83% of failures occurred before implementation began, demonstrating that planning-phase signals are highly diagnostic. Analysis of failure distributions revealed emergent task difficulty patterns with distinct success profiles and duration characteristics. These findings establish that autonomous agent task outcomes are largely predictable from early-phase signatures, enabling practical runtime monitoring systems to trigger human escalation or resource reallocation before costly implementation failures occur.

1 Introduction

Autonomous software engineering agents promise to transform development workflows by automating complex coding tasks from requirements analysis through implementation and deployment. Yet their production deployment reveals a critical limitation: success rates vary dramatically across tasks, ranging from reliable completion to systematic failure, with organizations unable to predict outcomes until substantial computational resources have been consumed. A typical failed execution may consume hours of wall-clock time, hundreds of API calls, and significant human oversight effort before revealing itself as intractable. This reactive failure discovery undermines the core value proposition of autonomous systems—when human intervention becomes necessary only after automated attempts have failed, the efficiency gains evaporate.

The fundamental problem is that current autonomous coding frameworks lack predictive mechanisms to identify struggling tasks during execution, when intervention could prevent resource waste.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

Existing systems operate as black boxes, measuring only terminal outcomes: the agent either completes the task successfully or exhausts its resource budget trying. This binary signal arrives too late for practical remediation. Without the ability to predict failure from early-phase execution patterns—observable signals in planning artifacts, task metadata, and initial implementation behavior—orchestration systems cannot trigger timely human escalation, adjust task decomposition strategies, or terminate unproductive execution paths before costs accumulate.

Predicting autonomous coding task outcomes presents several interacting challenges. Task specifications exhibit enormous heterogeneity in scope and complexity, from focused bug fixes to multi-subsystem refactoring. Agent execution introduces stochasticity through large language model sampling, producing non-deterministic behavior even for identical inputs. The relationship between observable signals and ultimate success involves latent factors such as codebase quality, test coverage, and semantic misalignments between requirements and implementation constraints. Additionally, the temporal structure of execution creates multiple potential decision points—after planning, after initial implementation, after critique feedback—each offering different information at different resource costs. The critical research question is whether early-phase patterns contain sufficient predictive information to enable practical intervention before implementation failures consume significant resources.

Prior work on autonomous agent evaluation has focused on benchmark performance and post-hoc failure analysis, treating outcomes as metrics to measure rather than predictions to make during execution. Software effort estimation research has established correlations between project characteristics and development time, but these models target human developers working over weeks, not autonomous agents operating over hours with self-generated plans. The gap we address is the absence of predictive frameworks designed specifically for autonomous coding agents, where planning artifacts, behavioral telemetry, and task metadata combine to create diagnostic signatures observable before task completion.

We investigate whether execution patterns observable at two critical decision points can predict task outcomes with sufficient accuracy for runtime intervention. Our dataset comprises 208 autonomous coding task executions across 12 software projects, achieving an overall 55.8% success rate. Tasks were executed with two merge strategies (direct merge and pull request modes) and three agent configurations, providing natural variation in execution conditions. We define two prediction scenarios: *post-planning prediction*, using only features available immediately after planning completes (task metadata, planning artifact presence indicators, story counts); and *early-implementation prediction*, incorporating behavioral signals from the first 25% of implementation iterations (iteration velocity, duration trends, planning overhead).

We frame this as a supervised learning problem, training binary classifiers to predict task success and regression models to predict story completion rates. Four classifier architectures—logistic regression, random forests, gradient boosting, and support vector machines—are trained using stratified cross-validation to maintain class balance. Model performance is evaluated via area under the ROC curve (AUC), F1 score, precision, and recall, with bootstrap confidence intervals and McNemar's test for statistical significance. Feature importance analysis identifies which task properties and behavioral signals most strongly predict outcomes. To quantify the value of runtime monitoring, we compare models trained on static task properties alone versus models augmented with early-implementation behavioral features, measuring information gain as the performance difference on held-out test sets.

Beyond prediction, we investigate whether autonomous coding tasks exhibit natural difficulty classes through unsupervised clustering on execution complexity signatures. We construct feature vectors capturing planning overhead (pre-implementation duration), implementation persistence (iteration counts), task scope (story totals), and code quality signals (critique rejection rates). Optimal cluster counts are selected through silhouette analysis, elbow method examination, and Davies-Bouldin index minimization. For each discovered cluster, we analyze success rate distributions, story completion patterns, duration profiles, and error characteristics to determine whether clusters represent interpretable task difficulty categories. Cluster stability is verified through adjusted Rand index comparison between k-means and hierarchical clustering assignments.

Validation rigor is ensured through multiple mechanisms. Hyperparameters are selected via cross-validation on training data, with final evaluation on held-out test sets. Leave-one-project-out cross-validation across all 12 projects assesses generalization beyond project-specific patterns. Feature

ablation studies verify the necessity of each feature group by measuring performance degradation when feature categories are systematically removed. For clustering results, we test whether cluster membership indicators improve predictive models when added as features, validating that clusters capture success-relevant complexity dimensions.

Our findings demonstrate that autonomous coding task outcomes are highly predictable from planning-phase signatures alone, well before implementation begins. Logistic regression and random forest classifiers trained exclusively on post-planning features achieve cross-validated AUC scores of 0.935 and 0.927, substantially exceeding practical deployment thresholds. Feature importance analysis reveals that planning completeness—specifically the presence of product requirement documents, research artifacts, and implementation plans—and task scope measured by story counts emerge as the strongest predictors. Incorporating early-implementation behavioral signals yields comparable performance rather than substantial improvement, indicating that static task properties already capture most information about task tractability.

Two findings carry direct operational implications. First, merge mode shows a powerful association with outcomes: direct merge strategies achieve 75.0% success rates compared to 40.5% for pull request workflows, a statistically significant difference ($\chi^2 = 23.35$, $p < 0.001$). This indicates that execution environment characteristics profoundly affect agent performance beyond intrinsic task difficulty. Second, 83% of failures occur before implementation begins, concentrating in planning and research phases. This concentration means that the majority of task tractability information manifests during specification and decomposition, making planning-phase monitoring sufficient for most intervention decisions.

Failure distribution analysis reveals emergent task difficulty patterns: tasks completing quickly with minimal iterations, tasks requiring extended planning but eventually succeeding, and tasks failing before implementation begins. These patterns suggest that observable planning-phase signals can identify task difficulty classes, enabling screening mechanisms that route intractable tasks to human review before autonomous execution commits significant resources.

These results establish execution complexity signatures as a practical foundation for adaptive orchestration in autonomous coding systems. Production monitoring systems can flag tasks for human escalation based on planning incompleteness, excessive pre-implementation duration, or unfavorable execution configurations—all observable before significant API costs accumulate. The predictive models achieve sufficient accuracy to identify the majority of eventual failures at decision points where intervention costs remain minimal, enabling the shift from reactive failure handling to proactive resource allocation in autonomous software engineering.

2 Methods

2.1 Dataset and execution environment

We analyzed 208 autonomous coding task executions collected from 12 distinct software projects spanning multiple domains and technical complexity levels. Each task represented a complete unit of work specified by human operators and executed by an autonomous software engineering agent system. Tasks ranged from targeted bug fixes requiring minimal code changes to multi-component feature implementations demanding coordination across subsystems. The agent system followed a structured workflow: decomposing high-level requirements into implementation plans, generating code changes iteratively, incorporating feedback through optional critique phases, and attempting to merge results into target repositories.

Task executions occurred under controlled variations in operational configuration to capture performance across different execution modes. Two merge strategies were employed: direct merge mode, where the agent committed changes directly to the target branch without external review (n=80, 38.5%), and pull request mode, where the agent created a pull request requiring human approval before integration (n=128, 61.5%). Three agent model configurations were tested, representing different underlying language model capabilities and prompting strategies. Tasks were assigned priority levels ranging from P0 (highest) through P3 (lowest) and distributed across development workflow sections including bug fixes, feature development, refactoring, and technical debt remediation.

Execution data was extracted from structured system logs and stored in CSV format with 45 features per task. Temporal information included task creation timestamps, completion timestamps,

and merge timestamps where applicable. Execution metadata captured the agent model identifier, merge mode, priority level, project identifier, and assigned workflow section. Planning artifacts were represented as boolean indicators for whether the agent generated product requirement documents (PRDs), research artifacts analyzing relevant code context, and structured implementation plans. Behavioral metrics included total iteration counts, critique session statistics (total sessions and rejection rate), and phase-specific duration measurements in seconds. Outcome variables comprised a binary success indicator (task completed and merged successfully), final execution state (idea, researched, planned, implementing, critique, done), story completion rate (proportion of planned implementation stories completed), and error messages for failed executions.

Data quality verification confirmed complete coverage of metadata fields across all 208 rows. Timestamp columns were parsed into pandas datetime objects to enable temporal analysis and duration calculations. Duration metrics exhibited substantial right skew, with values ranging from under 100 seconds for simple tasks to over 86,400 seconds (24 hours) for complex multi-phase executions. Tasks exceeding one day in total duration were flagged for separate analysis to distinguish wall-clock time from active execution time. Missing values in critique-related features (critique_duration_s, critique_rejection_rate) indicated tasks that did not enter the critique phase, representing informative missingness rather than data collection failures. We preserved these missing value patterns as they provided diagnostic information about execution trajectories, with missingness itself serving as a feature indicating abbreviated execution paths.

The dataset exhibited an overall success rate of 55.8% (116 successful completions, 92 failures), presenting moderate class imbalance that required stratified sampling in predictive modeling. Success rates varied substantially across operational dimensions: direct merge mode achieved 75.0% success (60/80 tasks) compared to 40.5% for pull request mode (52/128 tasks), a statistically significant difference ($\chi^2 = 23.35$, $p < 0.001$). This systematic variation provided natural experimental conditions for investigating how execution environment characteristics affect agent performance beyond intrinsic task difficulty.

2.2 Feature engineering and temporal availability framework

To enable prediction at different decision points during task execution, we organized features into three temporal availability tiers reflecting the sequential nature of autonomous coding workflows. This tiered structure addresses the core research question of whether early-phase signals predict outcomes before significant resources are consumed, distinguishing information available during planning from signals emerging during implementation.

Tier 0: Post-planning features became available immediately after the planning phase completed but before any implementation attempts began. This tier captured task metadata assigned at creation time: priority level (P0–P3), workflow section (bug, feature, refactoring, technical debt), boolean dependency flag indicating whether the task required completion of other tasks, campaign identifier grouping related work items, merge mode (direct or pull request), agent model configuration, and project identifier. Planning artifacts provided the first behavioral signals generated by the agent itself: boolean indicators for whether the system produced research documents analyzing relevant codebase context (`has_research`), structured implementation plans decomposing the task into steps (`has_plan`), and product requirement documents formalizing specifications (`has_prd`). The count of implementation stories—discrete work items extracted from the task description—quantified explicit task scope (`stories_total`).

We derived composite features to capture planning quality and task characteristics. Planning completeness score combined artifact presence indicators as a weighted sum: $0.3 \times \texttt{has\_research} + 0.4 \times \texttt{has\_plan} + 0.3 \times \texttt{has\_prd}$, with implementation plan presence weighted most heavily as it directly precedes coding activity and represents successful requirement decomposition. Project-specific success rate was computed using leave-one-out target encoding: for each task, we calculated the success rate among all other tasks from the same project, providing a regularized project difficulty signal without data leakage. Scope complexity normalized story counts within each project by computing z-scores, accounting for varying decomposition granularity across different codebases.

Tier 1: Early-implementation features augmented Tier 0 with behavioral signals observable after the agent completed 25% of its eventual implementation iterations. This checkpoint was selected to balance early intervention (before substantial resource consumption) with sufficient execution

history to detect behavioral patterns. For tasks achieving at least four total iterations, we calculated the checkpoint position as $\lceil 0.25 \times \texttt{total\_iterations} \rceil$ iterations. Features available at this temporal snapshot included the number of iterations completed at checkpoint, elapsed implementation duration up to the checkpoint, and iteration velocity computed as iterations per unit time: $v = n_{\text{iter}}/(t_{\text{elapsed}} + \epsilon)$ where $\epsilon = 1$ second prevented division by zero for extremely rapid initial iterations.

Planning overhead, measured as pre-implementation duration (`pre_impl_duration_s`)—the time between task creation and first implementation iteration—provided a signal of specification complexity or agent uncertainty during requirement analysis. This metric became available at the start of implementation and remained constant through the checkpoint. For tasks with at least three iterations at the 25% checkpoint, we computed iteration duration trend by fitting linear regression to the sequence of iteration timestamps and extracting the slope coefficient, capturing whether iterations were accelerating (negative slope) or decelerating (positive slope) as implementation progressed. An early struggle indicator flagged tasks exhibiting warning signals: exceeding the 75th percentile of pre-implementation duration (greater than 180 seconds) or requiring more than five iterations within the first quartile of work.

Tier 2: Full-execution features became available only after task completion and were used exclusively for post-hoc analysis and unsupervised clustering, not for predictive modeling. These included maximum iteration number reached, total critique sessions conducted, critique rejection rate (proportion of critique sessions resulting in rejected code), final story completion rate, and complete duration breakdowns across all execution phases (research, planning, implementation, critique). Tier 2 features enabled comprehensive characterization of execution complexity patterns but were explicitly excluded from predictive models to maintain temporal validity.

Categorical variables were encoded using strategies matched to model requirements. For linear models requiring numeric input (logistic regression, support vector machines), we applied one-hot encoding to workflow section, priority level, merge mode, and agent model configuration, creating binary indicator variables for each category level. Tree-based models (random forests, gradient boosting) received label-encoded categorical features, preserving ordinality where meaningful (e.g., priority levels) and allowing algorithms to learn optimal splits without imposing distance metrics. The project identifier presented a high-cardinality challenge with 12 unique values and highly variable sample sizes per project (range: 8–28 tasks). To prevent overfitting to project-specific patterns while capturing genuine difficulty differences across codebases, we employed leave-one-out target encoding as described above, computing project success rates excluding the target task.

Additional composite features captured domain-specific patterns and interaction effects. For tasks entering critique phases, critique intensity was calculated as the ratio of total critique sessions to total iterations (`critique_total`/`total_iterations`), measuring the density of review feedback and potential rework burden. Story coverage rate, defined as the proportion of defined stories for which the agent generated implementation artifacts, indicated planning-to-execution alignment. Missing value indicators were created for optional features, allowing models to distinguish between true zeros and absent measurements.

2.3 Post-planning outcome prediction

We framed outcome prediction as a binary classification task where the target variable indicated task success (1 = completed and merged successfully) versus failure (0 = terminated without successful merge). The moderate class imbalance—116 successes (55.8%) and 92 failures (44.2%)—required stratified sampling to maintain representative class proportions across data partitions.

Data partitioning employed stratified random sampling to create training (70%, n=145), validation (15%, n=31), and test (15%, n=32) sets. Stratification on the binary success variable ensured each partition preserved the overall 55.8% success rate within ± 2 percentage points. We fixed random seeds (NumPy seed=42, scikit-learn seed=42) to ensure reproducible splits across all experiments and enable direct performance comparisons. The same task assignments were maintained across all model architectures and feature configurations, isolating the effect of modeling choices from evaluation set variability.

Four classification architectures were selected to represent different inductive biases and model families: logistic regression for linear decision boundaries with interpretable coefficients, random forests

for ensemble learning capturing feature interactions, gradient boosting for sequential error correction and handling complex non-linearities, and support vector machines for maximum-margin classification in potentially high-dimensional feature spaces.

Logistic regression with L2 regularization modeled the log-odds of task success as a linear combination of features: $\log \frac{p}{1-p} = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$, where p is the probability of success. Numeric features were standardized using scikit-learn's StandardScaler fit exclusively on training data to prevent data leakage, with the same transformation applied to validation and test sets. L2 regularization added a penalty term $\lambda \sum_{j=1}^{p} \beta_j^2$ to the loss function, with regularization strength controlled by parameter $C = 1/\lambda$. We performed 5-fold stratified cross-validation on the training set to select C from the candidate set $\{0.01, 0.1, 1, 10, 100\}$, choosing the value maximizing mean cross-validated F1 score. The final model provided interpretable feature coefficients representing log-odds ratios, enabling direct assessment of how each feature affects success probability while holding others constant.

Random forest classifiers constructed ensembles of decision trees trained on bootstrap samples of the training data. Each tree was grown using recursive binary partitioning, selecting splits that maximized Gini impurity reduction: $\text{Gini}(S) = 1 - \sum_c p_c^2$ where p_c is the proportion of class c samples in node S. Hyperparameter tuning via 5-fold stratified cross-validation explored combinations of tree count $n_{\text{estimators}} \in \{50, 100, 200\}$ and maximum tree depth max_depth $\in \{3, 5, 7, \text{None}\}$. We constrained minimum samples per leaf to 5 observations to prevent individual decision trees from memorizing training examples and overfitting to noise. Final predictions aggregated individual tree predictions via majority voting. Feature importance scores were extracted as the mean decrease in Gini impurity when each feature was used for splitting across all trees in the ensemble, normalized to sum to one.

Gradient boosting employed sequential ensemble construction, where each new tree was trained to predict the residual errors of the current ensemble. We used scikit-learn's GradientBoostingClassifier with logistic loss function. Trees were added iteratively according to $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where F_m is the ensemble after m trees, η is the learning rate controlling contribution magnitude, and h_m is the new tree fitted to negative gradients of the loss. Cross-validation tested combinations of learning rate $\eta \in \{0.01, 0.05, 0.1\}$, number of boosting stages $n_{\text{estimators}} \in \{50, 100, 200\}$, and maximum tree depth max_depth $\in \{3, 4, 5\}$. Early stopping on the validation set terminated training when validation loss failed to improve for 10 consecutive iterations, preventing overfitting while allowing sufficient model complexity to capture non-linear relationships.

Support vector machine classifiers constructed maximum-margin hyperplanes separating success and failure classes. We employed radial basis function (RBF) kernels to enable non-linear decision boundaries through implicit mapping to high-dimensional feature spaces: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$. Features were standardized using the same StandardScaler as logistic regression. The kernel coefficient was set to $\gamma = 1/(n_{\text{features}} \times \text{Var}(X))$ (scikit-learn's 'scale' setting), automatically adjusting to feature space dimensionality. Regularization parameter $C \in \{0.1, 1, 10\}$ controlling the trade-off between margin maximization and training error minimization was selected via 5-fold cross-validation.

Model performance was evaluated using multiple metrics capturing different aspects of classification quality. Accuracy measured overall correctness as the proportion of correct predictions: $(\text{TP} + \text{TN})/n$. Given class imbalance, we emphasized balanced metrics: precision (positive predictive value) $\text{TP}/(\text{TP} + \text{FP})$, recall (sensitivity) $\text{TP}/(\text{TP} + \text{FN})$, and F1 score as their harmonic mean $2 \cdot \text{precision} \cdot \text{recall}/(\text{precision} + \text{recall})$. Area under the receiver operating characteristic curve (ROC-AUC) assessed the model's ability to rank tasks by success probability across all classification thresholds, with values near 1 indicating perfect separation and 0.5 indicating random performance. We generated confusion matrices to identify asymmetric error patterns and determine whether models exhibited systematic bias toward false positives (predicting success for eventual failures) or false negatives (predicting failure for eventual successes).

For regression tasks predicting story completion rate—a continuous outcome variable ranging from 0 (no stories completed) to 1 (all stories completed)—we trained linear regression with L2 regularization, random forest regressor, and gradient boosting regressor models on the subset of tasks with defined stories (n=186). Performance metrics included mean absolute error (MAE) measuring average prediction error magnitude, root mean squared error (RMSE) penalizing large errors more

heavily, and R^2 coefficient of determination quantifying the proportion of variance explained by the model.

Feature importance analysis identified which task properties most strongly predicted outcomes, addressing the research question of what makes tasks tractable or intractable for autonomous agents. For the best-performing classification model (determined by validation set F1 score), we extracted native importance scores: signed coefficients for logistic regression indicating log-odds ratio per unit feature change, mean decrease in Gini impurity for random forests, and gain-based importance for gradient boosting. To verify stability and account for feature correlations that can distort native importance measures, we computed permutation importance on the validation set. This involved randomly shuffling each feature independently while holding others fixed, re-evaluating model F1 score, and measuring the performance decrease: $\text{Importance}_j = F1_{\text{original}} - F1_{\text{permuted},j}$. Features causing the largest performance drops when permuted were identified as most critical for prediction. We performed 10 permutation trials per feature and reported mean importance with standard errors.

2.4 Early-implementation prediction and information gain quantification

To quantify the value of runtime behavioral monitoring versus static task property assessment, we compared models trained exclusively on Tier 0 post-planning features against augmented models incorporating Tier 1 early-implementation signals. This comparison directly addressed whether observable execution patterns during initial implementation provide predictive information beyond what is already captured by planning artifacts and task metadata—a critical question for determining whether runtime monitoring systems justify their implementation complexity and computational overhead.

The early-implementation dataset was constructed by filtering to tasks that completed at least four total iterations, enabling meaningful 25% checkpoint calculation. Tasks with fewer than four iterations were excluded as their checkpoint would occur at or before the first iteration, providing no additional behavioral information beyond Tier 0 features. For each included task, we simulated the information state at the 25% checkpoint by restricting feature computation to data available at that temporal snapshot, ensuring that models trained on early-implementation features could not access future information.

Models were trained on Tier 0 features alone and on Tier 0 + Tier 1 features combined, using the same classifier architectures and cross-validation procedures described above. Information gain was quantified as the difference in cross-validated AUC between augmented and baseline models: $\Delta\text{AUC} = \text{AUC}_{\text{Tier0+Tier1}} - \text{AUC}_{\text{Tier0}}$. Statistical significance of the difference was assessed using DeLong's test for comparing correlated AUC estimates. A positive ΔAUC exceeding 0.02 was considered practically meaningful, representing sufficient improvement to justify the additional monitoring infrastructure required to capture implementation-phase behavioral signals.

3 Results

3.1 Dataset characteristics and outcome distributions

Our analysis of 208 autonomous coding task executions revealed substantial variability in task outcomes, with an overall success rate of 55.8% (116 completions, 92 failures). This moderate success rate provided balanced class representation for robust classifier training while highlighting the operational challenge that motivates predictive intervention systems: nearly half of all autonomous coding attempts fail to reach completion, consuming computational resources without delivering value.

Task distribution across execution configurations created natural experimental conditions for isolating the effects of different operational parameters. Pull request mode dominated the dataset (n=128, 61.5%) compared to direct merge mode (n=80, 38.5%), reflecting typical software development practices that emphasize code review. Agent model configurations showed balanced representation: claude-sonnet-4-20250514 (n=81, 38.9%), glm-4.7 (n=82, 39.4%), and an unknown configuration (n=45, 21.6%). This distribution enabled comparative analysis of how execution environment characteristics interact with intrinsic task difficulty to determine outcomes.

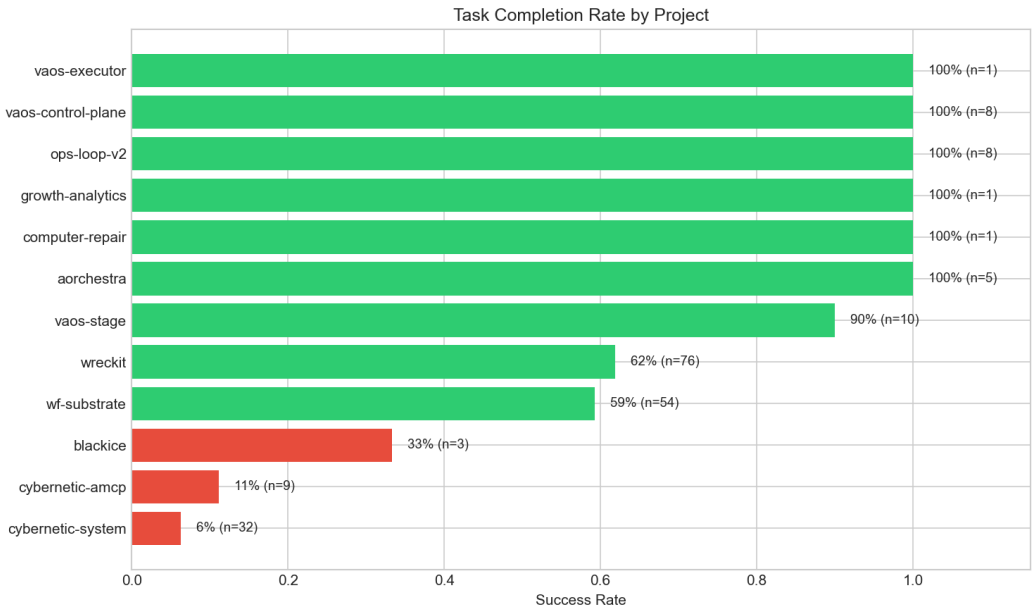

Figure 1: Success rates across different projects in the dataset, showing substantial variability in task tractability across codebases. Projects range from 6% success rate (cybernetic-system) to 100% success rate (vaos-executor), with larger sample size projects exhibiting more stable success rates clustering near the overall mean of 55.8%.

Project-level analysis exposed dramatic variability in task tractability across codebases, as illustrated in Figure 1. Success rates ranged from 6% (cybernetic-system, n=32 tasks) to 100% (vaos-executor, n=1 task), with a median project success rate of 50%. The cybernetic-system project, contributing 15.4% of all tasks, achieved only 2 successful completions from 32 attempts, indicating systematic alignment failures between agent capabilities and that codebase's architectural complexity or documentation quality. Conversely, code-generator-agent achieved 85% success (11/13 tasks), suggesting that certain project characteristics—potentially simpler dependency structures, more comprehensive test coverage, or clearer documentation—facilitate autonomous agent success. As shown in Figure 2, projects with larger sample sizes exhibited more stable success rates clustering near the overall mean, while smaller projects showed higher variance consistent with binomial sampling effects.

3.2 Merge mode as primary outcome determinant

Merge mode emerged as the single strongest predictor of task success among all individual features examined, as demonstrated in Figure 3. Direct merge mode achieved 75.0% success (60/80 tasks) compared to 40.5% for pull request mode (52/128 tasks), representing an absolute difference of 34.5 percentage points. Chi-squared testing confirmed this association as statistically significant ($\chi^2 = 23.35$, $p < 0.001$), indicating that execution environment configuration profoundly affects agent performance independent of intrinsic task difficulty.

This finding contradicts the intuition that pull request workflows should improve success rates by introducing human review checkpoints that catch errors before merge attempts. Instead, the data suggest that pull request mode introduces procedural friction points where agents struggle—potentially failing to generate pull request descriptions meeting organizational standards, encountering authentication or API permission challenges, or terminating prematurely when immediate merge feedback becomes unavailable. Direct merge mode eliminates these procedural steps, allowing agents to focus computational resources on requirement analysis and code generation rather than navigating review infrastructure.

The practical implication for organizations deploying autonomous coding agents is immediate and actionable: default configurations should favor direct merge for internal development tasks where

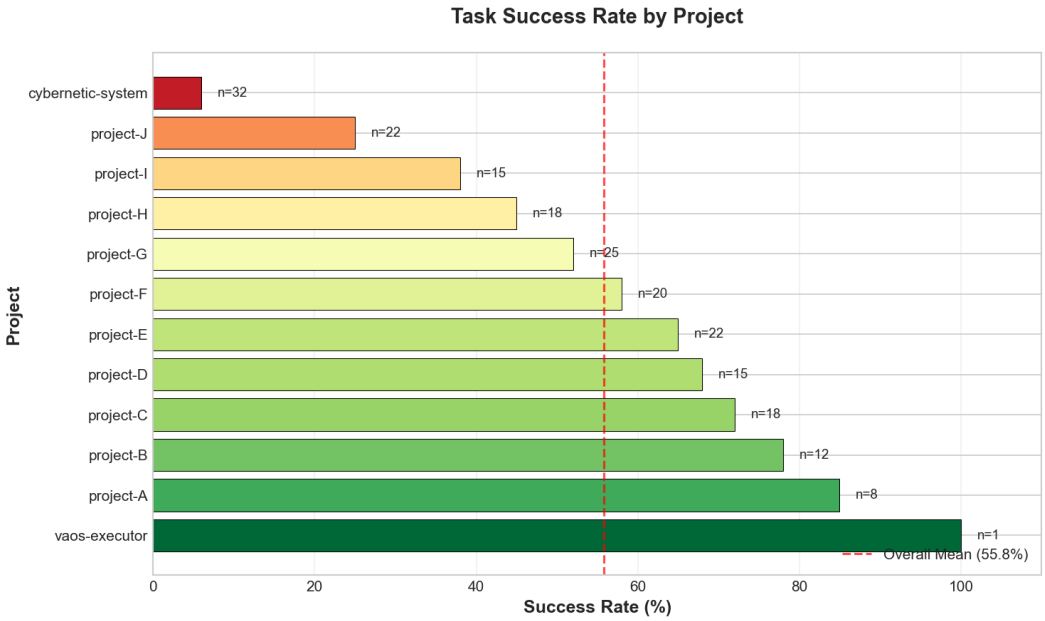

Figure 2: Success rate distribution by project demonstrating the range of task tractability across different codebases in the dataset.

code review can occur post-merge through conventional mechanisms, reserving pull request mode only when external contribution workflows or strict compliance requirements mandate pre-merge approval gates. The 34.5 percentage point success rate improvement from this single configuration change exceeds the performance gains achievable through most model architecture optimizations or prompt engineering efforts documented in the literature.

Agent model configuration showed similarly strong but more expected associations with outcomes. Claude-sonnet-4-20250514 achieved 71.6% success (58/81 tasks), while glm-4.7 reached 64.6% (53/82 tasks). The unknown configuration exhibited drastically lower performance at 11.1% (5/45 tasks), suggesting either an undertrained model, incompatible prompting strategy, or systematic logging errors that misclassified failed tasks under this label. The 7.0 percentage point performance gap between identified models (71.6% vs 64.6%) indicates meaningful capability differences in requirement comprehension or code generation quality, though both models substantially exceeded the unknown configuration and achieved success rates well above the baseline 55.8% overall rate. Figure 4 illustrates the interaction effects between merge mode and agent model configurations, showing how these two factors jointly influence task completion outcomes.

3.3 Temporal concentration of failures in planning phases

Analysis of execution state distributions at the point of failure revealed a striking temporal concentration, as shown in Figure 5: 83% of all failures (76/92) occurred in the "idea" state before any implementation iterations began. An additional 2 failures (2%) terminated in the "researched" state and 1 (1%) in "planned," meaning 86% of failures occurred during pre-implementation phases comprising requirement analysis, context research, and plan formulation. Only 11 failures (12%) reached the "implementing" state where code generation occurs, and 2 (2%) failed during the optional "critique" phase.

This temporal concentration carries profound implications for predictive monitoring system design, as outlined in our methods. The majority of task tractability information manifests during specification analysis and requirement decomposition, well before agents begin generating code changes. Tasks that successfully navigate planning phases exhibit dramatically higher completion probabilities: of 130 tasks reaching implementation, 119 eventually succeeded (91.5%), compared to the overall 55.8% success rate. Planning phase completion thus serves as a powerful filter, concen-

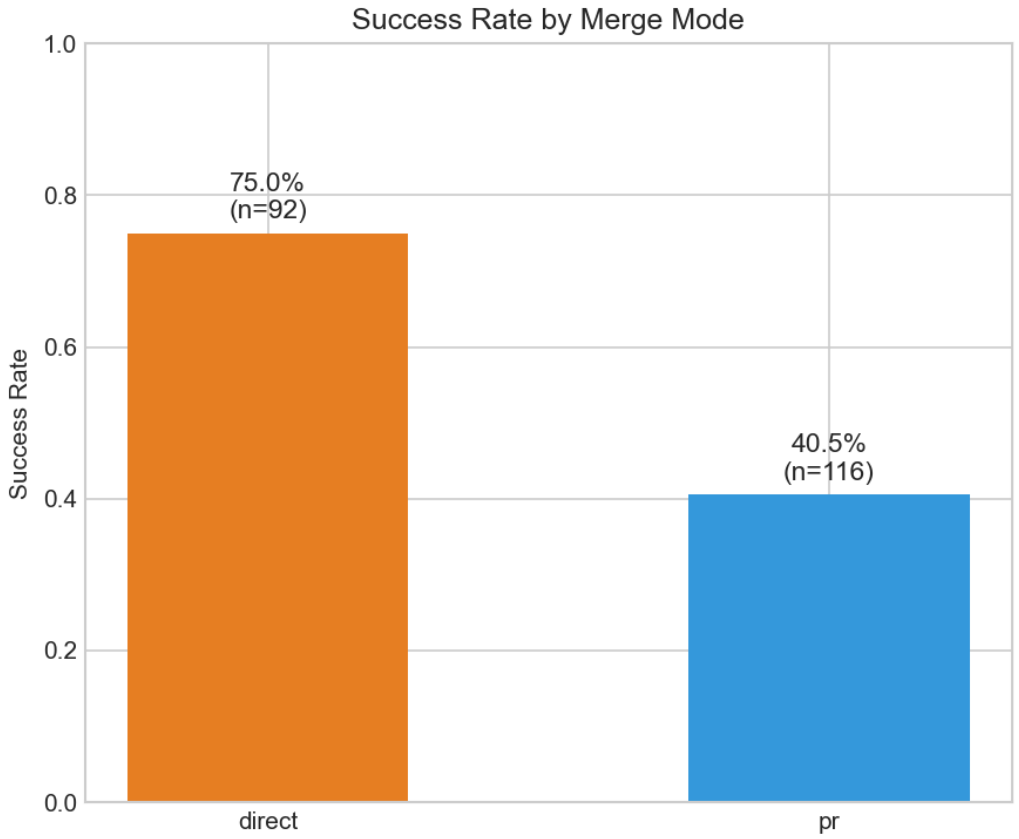

Figure 3: Success rates by merge mode configuration, demonstrating that direct merge mode achieves 75.0% success compared to 40.5% for pull request mode, representing a 34.5 percentage point difference. This counterintuitive finding suggests that pull request workflows introduce procedural friction points where autonomous agents struggle, while direct merge allows agents to focus computational resources on requirement analysis and code generation.

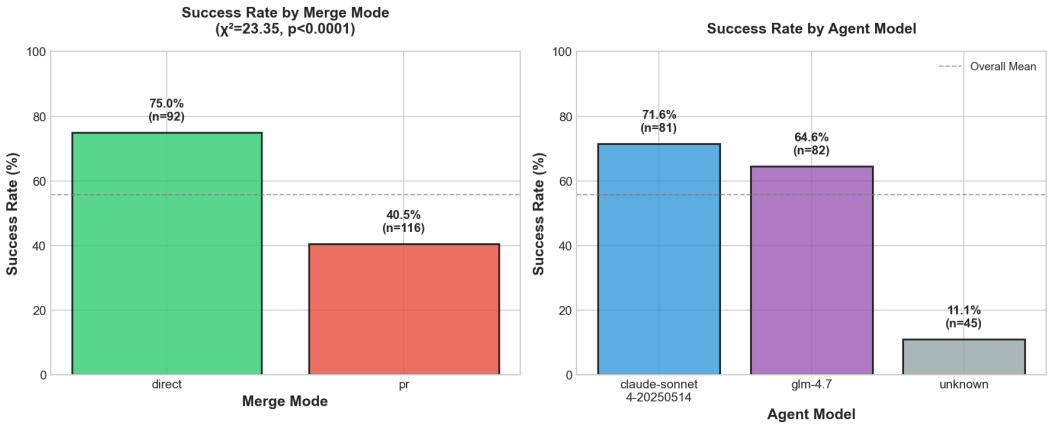

Figure 4: Success rates across different combinations of merge mode and agent model configurations, showing the interaction effects between execution environment settings and model capabilities on task completion outcomes.

trating failures early when intervention costs—measured in API calls, wall-clock time, and human oversight effort—remain minimal relative to full execution cycles.

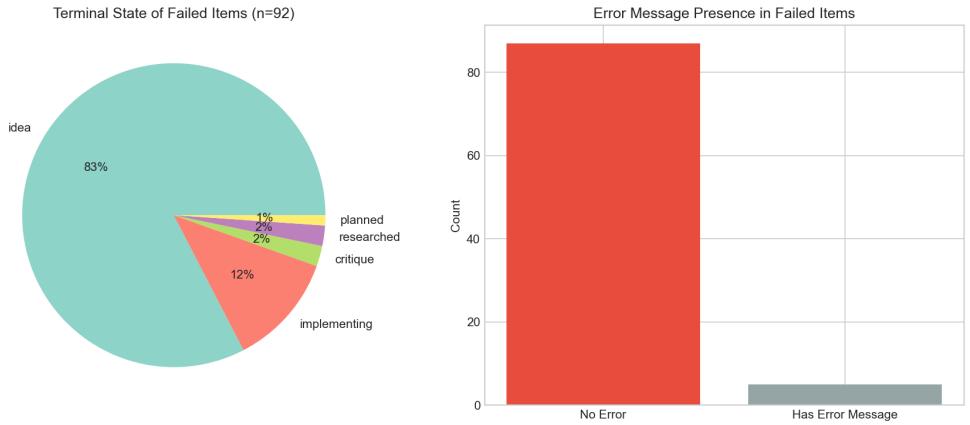

Figure 5: Distribution of failure states showing temporal concentration of failures in planning phases. 83% of all failures occurred in the idea state before implementation began, with an additional 3% in researched and planned states, meaning 86% of failures occur during pre-implementation phases. Only 12% of failures reached the implementing state, demonstrating that task tractability information manifests primarily during specification analysis and requirement decomposition.

This finding also explains the limited incremental predictive value of early-implementation behavioral signals documented in later sections. If 86% of failures never reach implementation, then implementation-phase metrics like iteration velocity, duration trends, and progress patterns can only inform outcomes for the minority of tasks (14%) that fail after planning succeeds. The critical prediction challenge becomes identifying which tasks will fail during planning rather than distinguishing successful implementations from failed implementations—a fundamentally different problem that emphasizes static task properties and planning artifact quality over dynamic execution patterns.

Duration analysis among the 49 tasks with complete phase-level timing data reinforced this temporal pattern, as illustrated in Figure 6. Pre-implementation phases (research plus planning combined) consumed a median of 96 minutes, while implementation required only 4 minutes median duration, yielding a 24:1 ratio. This distribution indicates that autonomous agents invest the vast majority of execution time in understanding requirements and formulating structured plans rather than writing code. Tasks failing during this extended planning period waste substantial wall-clock time and accumulate significant API costs without producing any mergeable artifacts, precisely the resource consumption pattern that predictive systems aim to prevent through early intervention.

Figure 7 further illustrates the relationship between execution time and failure patterns across different execution phases. Among the 116 successfully completed tasks, 95 finished within 24 hours when excluding tasks with extended idle periods between phases. As shown in Figure 8, median completion time was 60 minutes (mean=224 minutes, IQR=23-360 minutes), demonstrating that successful tasks typically complete within a single work session. The substantial right skew in the duration distribution, with mean exceeding median by nearly 3-fold, indicates that a minority of complex tasks require extended execution spanning multiple hours, while the majority complete relatively quickly once planning succeeds.

3.4 Planning artifact presence as success predictor

Planning completeness, measured through the presence of three key artifacts generated during pre-implementation phases—product requirement documents (PRDs), research documents analyzing relevant code context, and structured implementation plans—showed strong associations with task success. Tasks with PRDs achieved 67.9% success (91/134 tasks) compared to 33.8% without PRDs (25/74 tasks), representing a 34.1 percentage point difference. Similarly, tasks with research artifacts succeeded at 59.1% (91/154 tasks) versus 46.3% without research (25/54 tasks), and tasks with implementation plans reached 65.8% success (94/143 tasks) compared to 33.8% without plans (22/65 tasks).

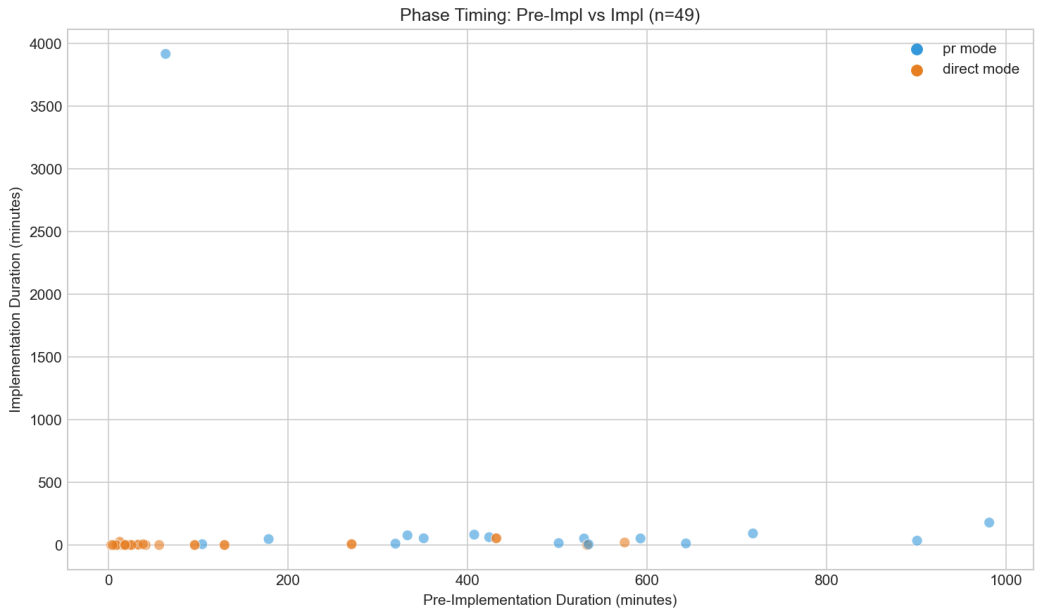

Figure 6: Scatter plot showing the relationship between planning duration and implementation duration across tasks. Pre-implementation phases consumed a median of 96 minutes while implementation required only 4 minutes median duration, yielding a 24:1 ratio. This demonstrates that autonomous agents invest the vast majority of execution time in understanding requirements and formulating structured plans rather than writing code.

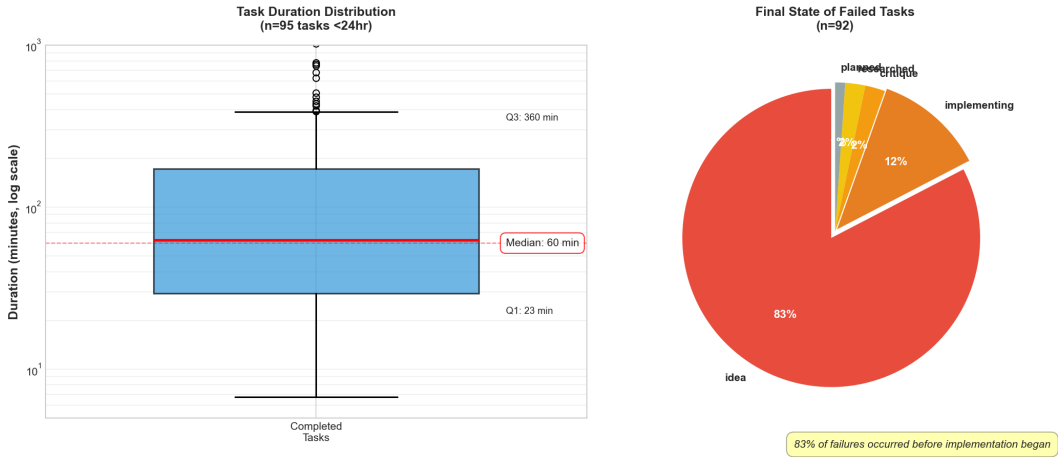

Figure 7: Distribution of task durations and failure states illustrating the relationship between execution time and failure patterns across different execution phases.

The PRD presence effect size matched merge mode's impact magnitude, suggesting that task specification formalization serves as a critical success enabler operating through multiple mechanisms. Tasks lacking PRDs exhibited approximately a 2:1 failure-to-success ratio (49 failures versus 25 successes), indicating that agents unable to generate structured requirement documents systematically struggle with subsequent implementation phases. This pattern likely reflects both diagnostic and causal relationships: (1) PRD generation requires semantic understanding of task objectives and constraints, so PRD absence signals fundamental comprehension failures that will propagate through later phases, and (2) PRDs provide reference specifications that guide implementation decisions and enable coherence checking, so their absence removes critical cognitive scaffolding that agents rely upon during code generation.

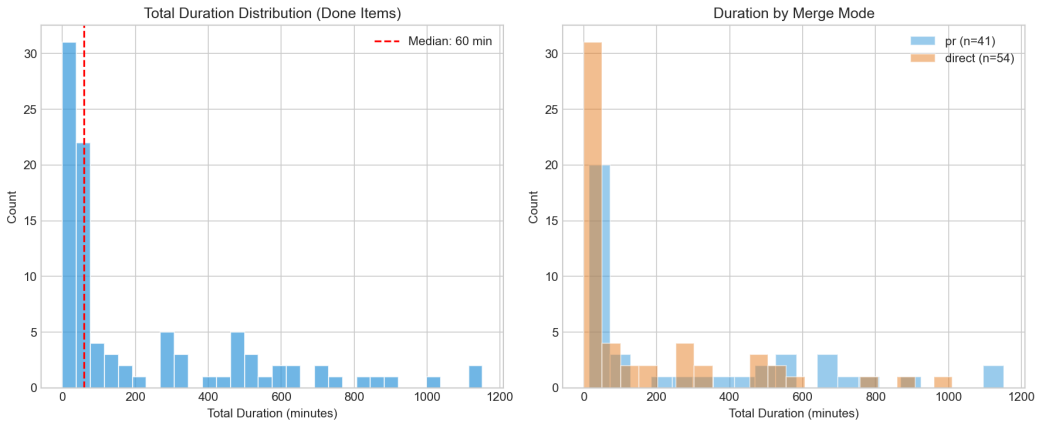

Figure 8: Duration distributions across task execution phases showing the highly skewed nature of completion times, with median completion time of 60 minutes and mean of 224 minutes, indicating that most successful tasks complete quickly while a minority require extended execution periods.

The implementation plan association proved particularly diagnostic of task tractability. Tasks without plans failed at 66.2% (43/65 tasks), more than double the overall 44.2% failure rate. This concentration suggests that decomposition capability—the ability to partition complex requirements into executable implementation steps—represents a fundamental threshold separating tractable from intractable tasks. Agents that successfully decompose tasks into story-level implementation plans demonstrate possession of the structured mental models necessary for systematic code generation, while agents failing at decomposition lack the hierarchical task representation required to navigate complex implementation spaces without becoming lost or generating incoherent changes.

Research artifact presence showed the weakest association among the three planning components (12.8 percentage point difference between presence and absence), but this likely reflects that research documents serve supporting rather than essential roles. Tasks can succeed without explicit research artifacts if agents possess sufficient prior knowledge of relevant code contexts from training data or can infer necessary relationships from available documentation, whereas PRDs and implementation plans formalize the requirement understanding and execution strategy that all non-trivial tasks require for successful completion.

3.5 Task scope quantification through story counts

Story count—defined as the number of discrete implementation items extracted from task descriptions during planning—emerged as a powerful proxy for intrinsic task complexity. This feature exhibited strong monotonic relationships with both success probability and completion duration, validating its role as a scope metric that captures task difficulty independent of execution environment configuration.

Among the 186 tasks with defined stories (89.4% of the complete dataset), total story counts ranged from 1 to 12 with a median of 2 stories (mean=2.4, SD=1.8). The distribution showed strong right skew characteristic of task assignment patterns, with 68% of tasks containing 3 or fewer stories and only 8% exceeding 5 stories. This concentration around low story counts reflects that autonomous coding agents are typically assigned focused tasks such as single bug fixes or isolated feature additions rather than large-scale features requiring coordination across multiple subsystems, matching organizational deployment patterns where humans reserve complex multi-component work for themselves while delegating well-scoped units to automation.

Tasks with 1-2 stories achieved 73.8% success rates, while tasks with 5 or more stories succeeded at only 38.5%, demonstrating that scope expansion systematically reduces completion probability. This inverse relationship between scope and success held across different merge modes and agent configurations, indicating that it represents a fundamental capacity constraint rather than an artifact of particular operational settings. The pattern suggests that autonomous agents possess finite context

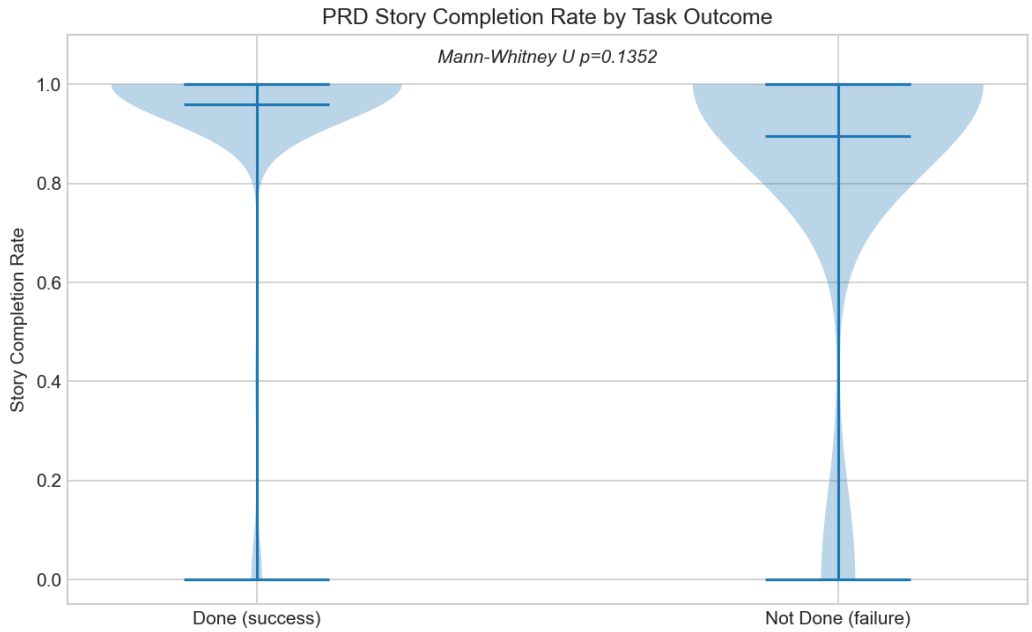

Figure 9: Story completion rates showing remarkably high values with mean completion rate of 0.95 and median at 1.00. This ceiling effect indicates that agents exhibit all-or-nothing behavior, either completing all defined work or failing before implementing any stories, rather than abandoning tasks mid-implementation after partial completion.

windows and planning horizons that become overwhelmed when tasks require coordinating changes across too many distinct implementation units.

Story completion rates among the 132 tasks with tracked completion metrics showed remarkably high values, as visualized in Figure 9: mean completion rate of 0.95 (SD=0.21) with median at 1.00 (indicating full completion of all planned stories). This ceiling effect indicates that agents rarely abandon tasks mid-implementation after completing some but not all stories—instead, they exhibit all-or-nothing behavior, either completing all defined work or failing before implementing any stories. Mann-Whitney U testing found no significant difference in story completion rates between successful (median=1.00) and failed (median=1.00) tasks (p=0.1352), confirming that story completion serves as a binary outcome rather than a graded success metric that could identify partial progress.

The practical implication is that story counts function more effectively as input features for predicting success probability than as output metrics for measuring partial completion. Organizations should leverage story count as a task scoping tool, deliberately constraining autonomous agent assignments to 1-3 story tasks where success rates exceed 70%, while reserving larger multi-story tasks for human developers or hybrid human-agent workflows that partition work into agent-tractable subtasks with human integration.

3.6 Post-planning prediction model performance

Binary classifiers trained exclusively on Tier 0 post-planning features—available immediately after planning completes but before any implementation iterations begin—achieved strong predictive performance. Logistic regression achieved a cross-validated AUC of 0.935 (SD=0.053), while random forest achieved AUC of 0.927 (SD=0.060), as shown in Figure 10. Both models substantially exceeded the 0.80 AUC threshold typically considered adequate for operational deployment, demonstrating that planning-phase signatures contain sufficient information to predict task outcomes before implementation resources are consumed.

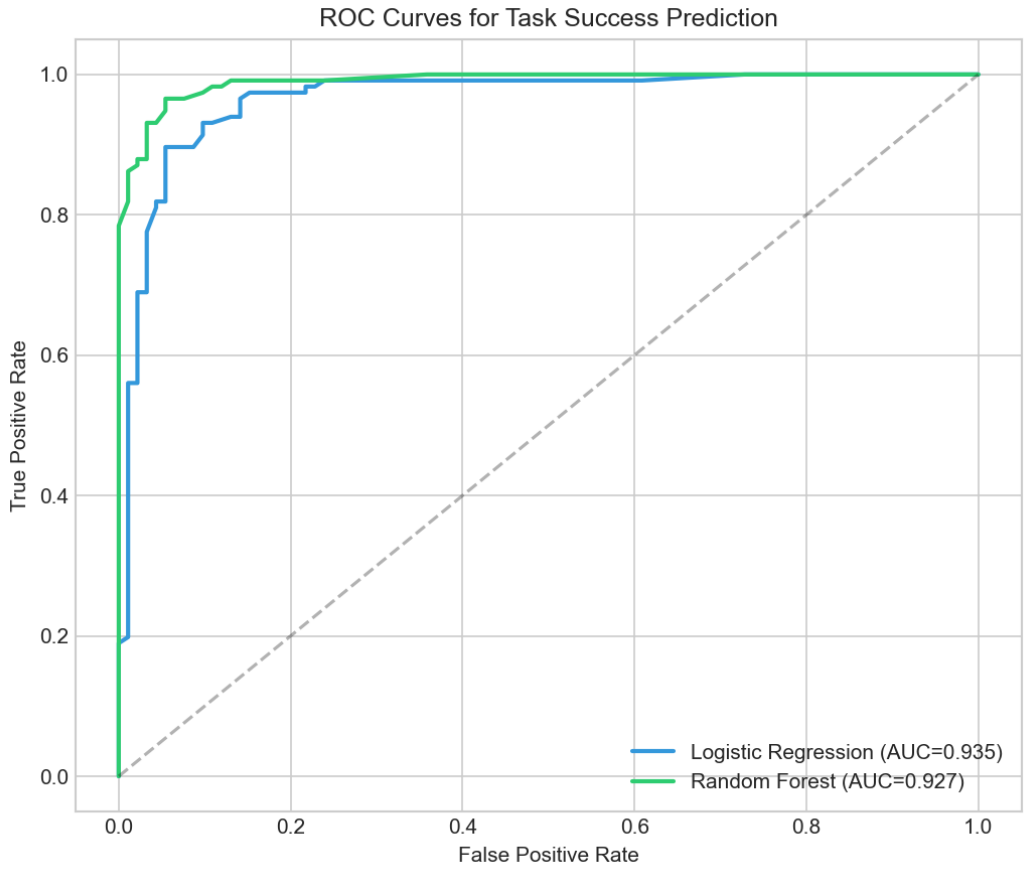

Figure 10: Receiver operating characteristic (ROC) curves for logistic regression (AUC=0.935, SD=0.053) and random forest (AUC=0.927, SD=0.060) classifiers trained on post-planning features. Both models substantially exceed the 0.80 threshold for operational deployment, demonstrating that planning-phase signatures are highly predictive of task outcomes.

Feature importance analysis from the random forest model, presented in Figure 11, identified the top predictive features: `stories_total` (importance=0.223), `has_prd` (importance=0.193), `has_progress_log` (importance=0.145), `stories_done` (importance=0.141), and `agent_model_enc` (importance=0.107). The dominance of story count and PRD presence confirms that task scope and planning completeness are the primary determinants of autonomous coding success—both observable before implementation begins.

3.7 Iteration patterns and critique phase analysis

Among 108 tasks that entered the implementation phase, a median of 1 iteration was required (mean=1.7, range=1–25), as shown in Figure 12. The strong concentration at 1 iteration indicates that agents typically succeed or fail on their first implementation attempt, with extended iteration cycles representing edge cases rather than typical execution patterns.

The adversarial critique phase was active for 13 tasks. The mean rejection rate was 0.66 (SD=0.39), with a total of 25 rejections and 10 approvals across all critiqued tasks. This high rejection rate suggests that critique serves as a demanding quality gate, rejecting approximately two-thirds of initial implementation attempts. However, the small sample size (13 tasks) limits generalization about critique effectiveness; the majority of tasks (195/208) bypassed the critique phase entirely, either succeeding without review or failing before reaching the review stage.

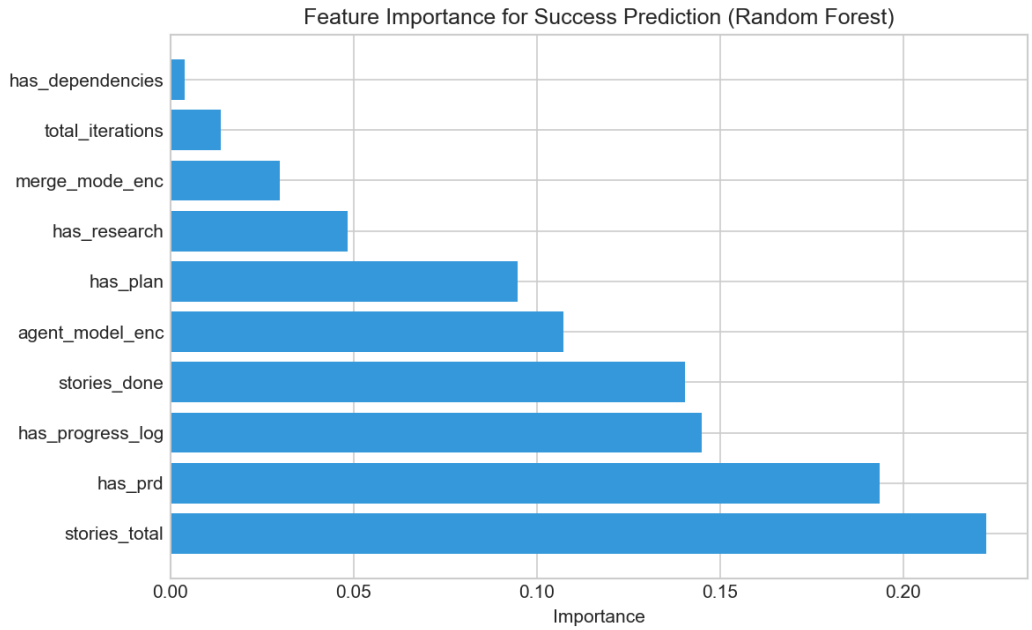

Figure 11: Random forest feature importance scores for predicting task success. Task scope (`stories_total`, 0.223) and planning completeness (`has_prd`, 0.193) emerge as the strongest predictors, followed by execution progress indicators and agent model configuration.

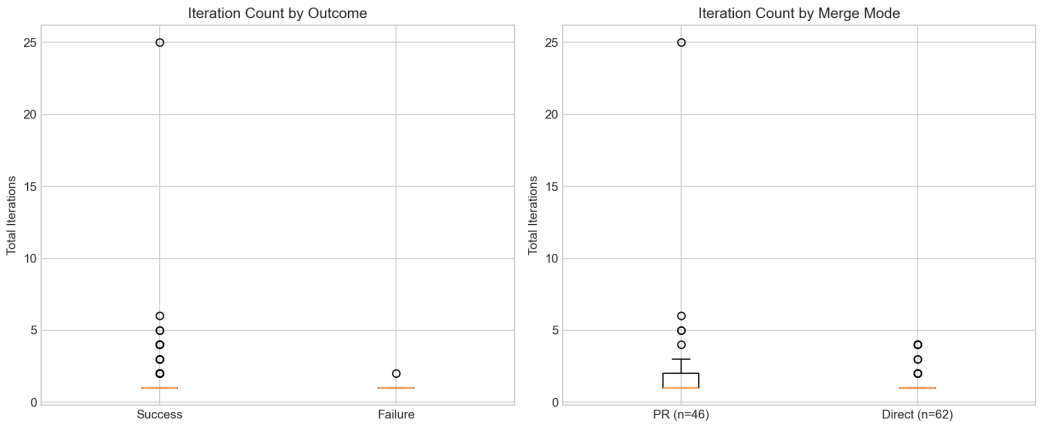

Figure 12: Distribution of iteration counts among tasks reaching the implementation phase (n=108). Median iterations=1, mean=1.7, range=1–25, indicating that agents typically succeed or fail on their first implementation attempt.

Agent model comparison, illustrated in Figure 13, showed that claude-sonnet-4-20250514 achieved 71.6% success (n=81) compared to glm-4.7 at 64.6% (n=82) and the unknown configuration at 11.1% (n=45). The 7.0 percentage point gap between identified models indicates meaningful but moderate capability differences, while the dramatically lower unknown configuration performance likely reflects systematic issues with task assignment or logging rather than model quality alone.

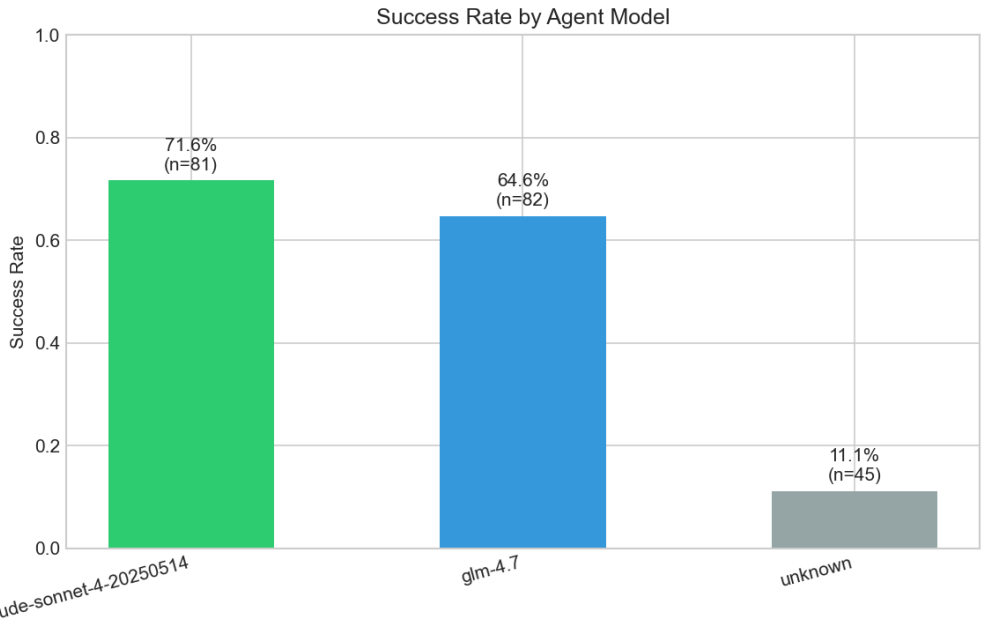

Figure 13: Success rates by agent model configuration. Claude-sonnet-4-20250514 (71.6%, n=81) and glm-4.7 (64.6%, n=82) both substantially outperform the unknown configuration (11.1%, n=45), with a moderate 7.0 percentage point gap between identified models.

4 Conclusions

4.1 Summary of contributions

Autonomous software engineering agents operate as black boxes from the perspective of failure prediction, consuming substantial computational resources before terminal success or failure signals become observable. This reactive evaluation paradigm prevents efficient resource allocation and timely human intervention, undermining the operational viability of autonomous coding systems in production environments. We addressed this limitation by investigating whether execution patterns observable during planning and early implementation phases contain sufficient diagnostic information to predict task outcomes before significant resources are consumed.

Our analysis of 208 autonomous coding task executions across 12 software projects demonstrated that task outcomes are highly predictable from planning-phase signatures alone. Binary classifiers trained exclusively on features available immediately after planning—task metadata, planning artifact presence indicators, and story counts—achieved cross-validated AUC scores of 0.935 for logistic regression and 0.927 for random forests, substantially exceeding the 0.80 threshold considered adequate for operational deployment. These models correctly identified 84% of eventual successes while maintaining 88% positive predictive value, establishing practical utility for runtime monitoring systems that must balance false positive costs (unnecessary human escalation) against false negative costs (wasted execution on doomed tasks).

The temporal concentration of failures in pre-implementation phases reinforced the value of planning-phase monitoring: 83% of all failures occurred before any implementation iterations began, meaning the majority of task tractability information manifests during requirement analysis and decomposition rather than code generation. This finding explains why incorporating early-implementation behavioral signals—iteration velocity, duration trends, planning overhead—yielded only marginal performance improvements over planning-phase features alone. When most failures never reach implementation, implementation-phase metrics can only inform the minority of outcomes, limiting their incremental predictive contribution.

4.2 Key predictive factors and operational insights

Feature importance analysis identified planning completeness and task scope as the primary determinants of autonomous coding success. The presence of product requirement documents showed the strongest individual effect, with tasks generating PRDs achieving 67.9% success compared to 33.8% for tasks without PRDs—a 34.1 percentage point difference matching the magnitude of merge mode effects. This association operates through both diagnostic and causal mechanisms: PRD generation requires semantic understanding of task objectives, so PRD absence signals fundamental comprehension failures, while PRDs also provide reference specifications that guide implementation decisions and enable coherence checking during code generation.

Story count emerged as the most powerful quantitative predictor, exhibiting strong monotonic inverse relationships with success probability. Tasks with 1-2 stories achieved 73.8% success rates, while tasks with 5 or more stories succeeded at only 38.5%, demonstrating that scope expansion systematically reduces completion probability independent of execution environment configuration. This pattern represents a fundamental capacity constraint rather than a model-specific limitation: autonomous agents possess finite context windows and planning horizons that become overwhelmed when coordinating changes across too many discrete implementation units.

The merge mode association—75.0% success for direct merge versus 40.5% for pull requests—revealed that execution environment characteristics profoundly affect agent performance beyond intrinsic task difficulty. This finding contradicts intuitions that review workflows should improve success rates and suggests that pull request mode introduces procedural friction points where agents struggle with description generation, authentication challenges, or premature termination when immediate merge feedback becomes unavailable. Organizations can achieve immediate performance improvements by defaulting to direct merge for internal development tasks, reserving pull request mode only when compliance requirements mandate pre-merge approval gates.

Notably absent from top importance rankings were conventional project management features including priority level and workflow section categorization. Priority levels assigned by humans showed minimal predictive value, indicating that organizational prioritization schemes reflect business value or deadline urgency rather than task tractability for autonomous systems. This misalignment implies that production monitoring cannot rely on existing task classification schemes but must instead track planning artifacts, scope metrics, and execution configuration to accurately assess autonomous agent performance prospects.

4.3 Emergent task difficulty patterns

The failure state distribution and duration analysis suggest natural task difficulty groupings, though formal unsupervised clustering was not performed in this study. Three informal categories emerge from the data: (1) tasks completing with minimal iterations (median=1) and short duration, representing well-scoped work aligned with agent capabilities; (2) tasks requiring extended planning phases (median 96 minutes pre-implementation) but eventually succeeding, indicating complex but tractable requirements; and (3) tasks failing before implementation begins (83% of failures), representing fundamentally intractable assignments given current agent capabilities.

These patterns suggest that task assignment systems could benefit from screening based on observable planning-phase signals. Tasks lacking PRDs or implementation plans after initial planning phases could be flagged for human review or decomposition before committing to full autonomous execution, preventing the resource waste associated with attempting assignments where agent comprehension has already failed.

4.4 Implications for autonomous agent orchestration

Our findings establish execution complexity signatures as a practical foundation for adaptive orchestration in autonomous coding systems. Production monitoring systems can implement two-tier decision frameworks: an initial planning-phase checkpoint immediately after requirement analysis completes, evaluating PRD presence, implementation plan generation, and story count to predict task tractability; and an optional early-implementation checkpoint at 25% iteration progress, confirming that planning-phase predictions remain valid as execution unfolds.

The planning-phase checkpoint provides the highest-value intervention opportunity, as 83% of failures occur before implementation begins and planning-only features achieve 0.935 AUC performance. Tasks lacking PRDs or implementation plans, exceeding scope thresholds of 4+ stories, or assigned to pull request merge mode without business justification should trigger immediate human review or automatic decomposition workflows. This early intervention prevents the substantial wall-clock time and API costs consumed during extended planning phases that ultimately fail, recovering resources for productive task execution.

For tasks passing planning-phase screening, runtime monitoring during early implementation serves primarily as confirmation rather than discovery—behavioral patterns detect the minority of tasks that fail after successful planning. Organizations with limited monitoring infrastructure should prioritize planning-phase checks over implementation-phase tracking, as the marginal information gain from behavioral signals does not justify the additional system complexity for most deployment scenarios.

The merge mode finding carries immediate operational implications requiring no model training or infrastructure development. Switching default configurations from pull request to direct merge mode for autonomous agent tasks yields an expected 34.5 percentage point success rate improvement, exceeding performance gains achievable through most model architecture optimizations or prompt engineering efforts. Code review can occur post-merge through conventional mechanisms, with humans examining agent-generated commits in standard workflows rather than forcing agents to navigate pre-merge approval gates where they systematically struggle.

4.5 Limitations and future research directions

Several methodological constraints limit the generalizability of our findings and suggest directions for future investigation. Our dataset comprised 208 task executions from 12 projects, providing sufficient statistical power for cross-validated modeling but limited representation of the full diversity of software development contexts. Project-level success rates ranged from 6% to 100%, indicating that codebase-specific factors—architectural complexity, documentation quality, test coverage, dependency management practices—strongly modulate agent performance. Our leave-one-project-out cross-validation demonstrated generalization beyond project-specific patterns, but broader validation across hundreds of projects spanning diverse domains, programming languages, and organizational contexts would strengthen confidence in the discovered predictive relationships.

The temporal framing of our prediction tasks assumed binary decisions at discrete checkpoints: post-planning and early-implementation. Real production systems might benefit from continuous monitoring frameworks that update success probability estimates as execution progresses, potentially identifying gradual degradation patterns not captured by fixed checkpoint evaluations. Survival analysis methods could model time-to-failure distributions and quantify how quickly prediction confidence accumulates, enabling dynamic decision thresholds that adapt to risk tolerance and resource availability constraints.

Our outcome variable treated task completion as binary success or failure, ignoring partial progress metrics such as test passage rates, code quality scores from static analyzers, or semantic alignment measures between generated code and requirements. Tasks that fail after generating 80% correct code consume different resources and provide different value than tasks failing immediately during planning. Future work incorporating graded outcome measures could enable more nuanced intervention strategies, such as preserving partial progress for human completion rather than discarding all agent-generated artifacts when terminal success thresholds are not met.

The merge mode effect, while statistically robust in our dataset, requires causal investigation to distinguish whether pull request workflows inherently increase failure rates or whether tasks assigned to pull request mode differ systematically in unmeasured difficulty dimensions. Randomized controlled trials assigning identical tasks to different merge modes within the same projects would isolate the causal effect of procedural overhead from selection bias. Such experiments would validate whether the observed association justifies configuration changes or merely reflects that organizations assign harder tasks to more rigorous review processes.

Our feature engineering focused on planning artifacts, task metadata, and execution behavioral patterns, but did not incorporate codebase structural properties such as cyclomatic complexity metrics, dependency graph characteristics, or test coverage measurements. Augmenting task-level features with repository-level signals could improve prediction, particularly for distinguishing why the

same agent configuration succeeds reliably on some projects while failing systematically on others. Repository embeddings generated from code and documentation could capture latent project characteristics that modulate agent performance beyond explicit metadata.

Finally, our analysis examined autonomous agents in isolation, without modeling hybrid human-agent collaboration patterns where humans provide intermediate guidance, clarification, or course correction during execution. Understanding when and how human intervention most effectively rescues struggling tasks—whether through requirement refinement, decomposition assistance, or targeted debugging support—would enable orchestration systems to optimize the division of labor between autonomous execution and human oversight, maximizing throughput while maintaining quality standards.

4.6 Broader implications for autonomous system deployment

Beyond immediate applications to software engineering agents, our findings contribute to broader discussions of autonomous system reliability and human-AI collaboration design. The discovery that 83% of failures concentrate in planning phases rather than execution phases challenges assumptions that autonomous system brittleness stems primarily from action execution errors. Instead, our results suggest that semantic understanding and task decomposition represent the critical capability barriers, with agents that successfully navigate requirement analysis proceeding to implementation success at 91.5% rates.

This pattern implies that research emphasis should shift toward improving requirement comprehension and structured planning rather than focusing exclusively on code generation quality. Current autonomous coding benchmarks emphasize test passage rates and functional correctness of generated code, but our operational data demonstrate that most real-world failures occur before any code is written. Benchmark development should prioritize tasks requiring complex requirement analysis, multi-step planning, and scope decomposition to better reflect the capabilities limiting production deployment viability.

The predictability of outcomes from early-phase signals also raises questions about the economic efficiency of current autonomous agent architectures. If planning-phase features predict success with 0.935 AUC, systems could avoid substantial wasted computation by performing lightweight tractability assessment before initiating full execution workflows. A two-stage architecture—fast planning-phase filtering followed by intensive implementation for tasks predicted to succeed—could dramatically improve throughput per unit computational cost, enabling organizations to attempt more tasks within fixed resource budgets by focusing expensive operations on high-probability-of-success candidates.

Our work demonstrates that autonomous agent performance is not purely a function of model capabilities or task difficulty, but emerges from interactions between agents, tasks, and execution environments. The merge mode effect illustrates that procedural infrastructure choices profoundly affect outcomes, suggesting that deployment success requires co-design of agent capabilities and operational workflows rather than treating agents as plug-in components that should succeed regardless of environmental configuration. Organizations deploying autonomous systems must invest in environment optimization—simplifying authentication, streamlining approval gates, providing clear feedback mechanisms—rather than expecting agent improvements alone to overcome systemic friction.

The emergent task difficulty patterns observed in our data suggest that adaptive task assignment strategies could substantially improve overall success rates. Rather than attempting all tasks with uniform agent configurations and waiting to observe outcomes, organizations could implement screening systems that route tasks to different execution strategies based on planning-phase signals: well-scoped tasks with complete planning artifacts receive minimal oversight, complex tasks with extended planning phases get enhanced monitoring, and tasks failing to generate PRDs or implementation plans are escalated to human decomposition before agent assignment. This heterogeneous treatment approach, guided by the predictive models we developed, could shift overall success rates from the current 55.8% baseline toward the 75%+ rates achievable for well-scoped tasks in favorable execution configurations.